ml 1 report

paul.kothgasser

April 2026

1 Linear Regression

1.1 Univariate Linear Regression

$$\mathcal{L}_U(\theta) = \frac{1}{N}\Sigma_{i=1}^{N}\left(f_\theta(x^i) - y^i\right)^2 \tag{1}$$

$$\theta^* = (b^*, w^*)^T \in argmin_\theta \mathcal{L}_U(\theta) \tag{2}$$

1.1.1 Exercise: derive a closed-form analytical solution for w^* and b^*

partial derivatives:

1. $\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N}\Sigma_{i=1}^{N}\frac{\partial}{\partial b}\left(b + wx^i - y^i\right)^2 = \frac{1}{N}\Sigma_{i=1}^{N}\frac{\partial}{\partial b}(b^2 + 2bwx^i - 2by^i + w^2(x^i)^2 - 2wx^iy^i + (y^i)^2) = \frac{1}{N}\Sigma_{i=1}^{N}2b + 2wx^i - 2y^i = \frac{2}{N}\left(\Sigma_{i=1}^{N}(b + wx^i - y^i)x^i\right)$

2. $\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{N}\Sigma_{i=1}^{N}\frac{\partial}{\partial w}\left(b + wx^i - y^i\right)^2 = \frac{1}{N}\Sigma_{i=1}^{N}\frac{\partial}{\partial w}(b^2 + 2bwx^i - 2by^i + w^2(x^i)^2 - 2wx^iy^i + (y^i)^2) = \frac{1}{N}\Sigma_{i=1}^{N}2bx^i + 2w(x^i)^2 - 2x^iy^i = \frac{2}{N}\left(b\Sigma_{i=1}^{N}x^i + w\Sigma_{i=1}^{N}x^i - \Sigma_{i=1}^{N}y^i\right) = \frac{2bN}{N} + \frac{2}{N}\left(\Sigma_{i=1}^{N}wx^i - y^i\right))$

Set them to 0, find expressions for the b and w that minimize. First, we know the derivative must vanish. from (2.) and defining $\bar{y} = \frac{1}{N}\Sigma_{i=1}^{N}y^i$ and $\bar{x} = \frac{1}{N}\Sigma_{i=1}^{N}x^i$, we can find an expression fo b, depending on w: $b^* = (\bar{y} - w\bar{x})$.
Using this expression in (1.) we get (neglecting the factor $\frac{2}{N}$, as it doesn't change whether the sum is zero): $\Sigma_{i=1}^{N}x^i(b^* + wx^i - y^i) = \Sigma_{i=1}^{N}x^i(\bar{y} - w\bar{x} + wx^i - y^i) = w\Sigma_{i=1}^{N}x^i(x^i - \bar{x}) - \Sigma_{i=1}^{N}(y^i - \bar{y})x^i$.
Now we know by definition of $\bar{x} = \frac{1}{N}\Sigma_{i=1}^{N}x^i$, that $\Sigma_{i=1}^{N}(x^i - \bar{x})x^i = N^2(\bar{x}^2 - \bar{x}^2) = 0 = N(\bar{x}^2 - 2\bar{x}\bar{x} + \bar{x}^2) = \Sigma_{i=1}^{N}(x^i - \bar{x})^2$ Setting the previous expression for w to zero continue: $w\Sigma_{i=1}^{N}(x^i - \bar{x})^2 - \Sigma_{i=1}^{N}(y^i - \bar{y})x^i = 0 \Rightarrow w\Sigma_{i=1}^{N}(x^i - \bar{x})^2 = \Sigma_{i=1}^{N}(y^i - \bar{y})x^i = \Sigma_{i=1}^{N}y^ix^i - \bar{y}\Sigma_{i=1}^{N}x^i = \Sigma_{i=1}^{N}y^ix^i - \frac{\Sigma_{i=1}^{N}y^i}{N}\Sigma_{i=1}^{N}x^i = \Sigma_{i=1}^{N}x^iy^i - \frac{\Sigma_{i=1}^{N}x^i\Sigma_{i=1}^{N}y^i}{N} + \Sigma_{i=1}^{N}x^iy^i - \frac{\Sigma_{i=1}^{N}x^i\Sigma_{i=1}^{N}y^i}{N} - \Sigma_{i=1}^{N}x^iy^i - \frac{\Sigma_{i=1}^{N}x^i\Sigma_{i=1}^{N}y^i}{N}$ (Productive zero) $= \Sigma_{i=1}^{N}x^iy^i - \Sigma_{i=1}^{N}x^i\bar{y} + N\bar{x}\bar{y} - \bar{x}\Sigma_{i=1}^{N}y^i = \Sigma_{i=1}^{N}(x^i - \bar{x})(y^i - \bar{y}) \Rightarrow w^* = \frac{\Sigma_{i=1}^{N}(x^i - \bar{x})(y^i - \bar{y})}{\Sigma_{i=1}^{N}(x^i - \bar{x})^2}$.

Thus

- $w^* = \frac{\Sigma_{i=1}^{N}(x^i - \bar{x})(y^i - \bar{y})}{\Sigma_{i=1}^{N}(x^i - \bar{x})^2}$
- $b^* = (\bar{y} - w\bar{x})$

1.1.2 programming exercise

1.1.3 programming exercise

1.1.4 discussion of plots and results

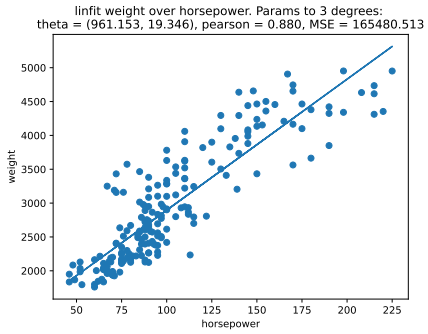


Figure 1: linear fit for data with roughly linear relationship

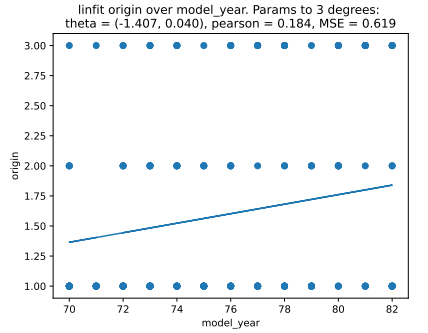


Figure 2: linear fit for data without linear relatinship

Regarding the scatter pots - one can see, that the fit tries to match general trends it notices as best as it can with only two parameters given. For linear or linearly approximable solutions, we can see that the function graph corresponds to the scattered position averaged out over displacements. When there is a clearly nonlinear behaviour, the graph fails to accurately model the response behaviour and does not depict the true relationship between data points at all.

This can, for example, be clearly seen in 2, as compared to 1.
regarding the Pearson coefficient - Since it is directly defined as correlation divided by the product of standard deviations, it can have any value between -1 and 1. The distance from zero in a positive direction measures the degree to which our data points are correlated, while the negative direction likewise measures its inverse (with a positive pearson coefficient, we get an increase in v2 when v1 rises, whereas a rising v1 decreases v2 for a negative one). The closer the coefficient lies to 1 or -1, the stronger the change in variables affects each other.

1.1.5 general discussion

```
Given a dataset D,
with w positive but almost zero.
Could we still observe a Pearson coefficient close to 1?
```

Yes, that is absolutely possible. The coefficient doesn't measure how strongly one variables increase (decrease) affects the change of the other function. Instead, it is more reasonably understood as a measure of how closely the two variables lie to a plotted linear function with the parameters specified in theta, ie. how closely the relationship in our data matches a simple linear function.

1.2 Multiple linear regression

1.2.1 model extension from linear to multivariate linear regression

The design matrix $\mathbf{X}$ should have N rows (one for each feature-target pair) and D+1 columns (one for each entry into θ). This ensures that $\mathbf{X}\theta$ has the same shape as y. The first column of $\mathbf{X}$ should be all ones, to contain the possible linear shift b, which is not further connected to x. The following rows should each contain x_1 to x_D. In a general sense, one could include non-linear functions of x in the matrix rows, to also model non-linear behaviour. This would be a step beyond the functionality required for this task and is thus only noted as an aside.
definition of $\mathbf{X}$

1. $\mathbf{X}$ is N x (D+1)
2. $\mathbf{X}_{ij} = x_j$
3. except $\mathbf{X}_{i0} = 1$

As for the derivative, we first need $||\mathbf{X}\theta - y||^2 = (\mathbf{X}\theta - y)^T(\mathbf{X}\theta - y)$. Then $\nabla_\theta \mathcal{L}_M(\theta) = \nabla_\theta(\frac{1}{N}(\mathbf{X}\theta - y)^T(\mathbf{X}\theta - y) =$ since y and $\mathbf{X}$ are independent of θ ie. their derivatives vanish $= \frac{1}{N}\left[\mathbf{X}^T(\mathbf{X}\theta - y) + (\mathbf{X}\theta - y)^T\mathbf{X}\right]$ Since the inner product is symmetric, we can express the last term as the previous one and get $\nabla_\theta \mathcal{L}_M = \frac{2}{N}\mathbf{X}^T(\mathbf{X}\theta - y)$.
Setting this to zero and adding $\mathbf{X}^T y$ to both sides of the equation, we get

$\mathbf{X}^T\mathbf{X}\theta = \mathbf{X}^T y \Rightarrow \theta^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T y$, with $(\mathbf{X}^T\mathbf{X})^{-1}$ also called left Moore-Penrose pseudoinverse.

1.2.2 programming exercise

1.2.3 programming exercise

For extending the linear fit of (displacement -¿ horsepower) to ([displacement, acceleration, weight] -¿ horsepower) to a three feature fit, we get the following results:

- $f_{\theta^*_M} = 102.119 + 0.105x_1 - 4.997x_2 + 2.020x_3$ with $(x_1, x_2, x_3) =$ (displacement, acceleration, weight)
- $\mathcal{L}_M(\theta_M) = 137.930$ compared to MSE of task 1.1 $\mathcal{L}_U(\theta^*_U) = 268.312$

1.2.4 Comment on statement

```
No matter how the data looks like (targets and features),
if theta M star is the minimizer for LM, our multiple linear regression,
and theta U star the minimizer for LU, our linear regression,
we will always have LM <= LU. That is, adding more fetures will never make the fit to th
```

This statement is correct, based on the following consideration:
First, we consider the case where the additional features do affect the function. clearly, a model that takes them into account is better suited to approximating the output function, thus the error would decrease.
Now consider adding a feature, which does not impact the output in the actual model. Since the input data doesn't affect the output data here, the minimizer will now set the weight of that feature to zero, essentially canceling its impact. This is also shown by the fact, that the linear fit is a special case of the multiple linear regression. As we find the optimal solution in both cases, either they coincide and both versions find the same optimum, or there is a more suitable solution found by the second method.
note, that this is a general statement, assuming we don't have a mistaken match between the feature and output data, which looks like a linear relationship. The issue of overfitting is not considered here.

1.3 Polynomial regression

1.3.1 design matrix for polynomial

We get $\mathbf{X}_{ij} = \phi_j(x_i)$ with $\phi_k(x) = x^k$. Then our matrix has the same row-column structure as above and is linear in ϕ_k which are non-linear in x. So

- $\mathbf{X}$ is N x (K+1)
- $\mathbf{X}_i j = \phi_j(x_i) = (x_i)^j$
- note that as $(x_i)^0 = 1$ we get $\mathbf{X}_i 0 = 1$ again

1.3.2 programming task

From visual comparison, the pair (displacement, horsepower) was chosen as (feature,target) pair. Also, for K the number 4 was chosen. The MSE achieved here was 214.125, which is a slight improvement over the linear MSE of 268.312.

1.3.3 small dataset, minimal K

Since we have 5 data points, theoretically a polynomial of degree 4 is able, to perfectly fit the data values. A polynomial of degree n is generally perfectly solvable for n +1 independent input-output pairs, with the extra pair being solved for by the $bx^0 = b$ component, aka. the bias. Thisd implies, that as long as the dimension of $\mathbf{X}$ (which is N by K+1) is equal to or larger than the number of data points, we can have a solution with loss zero on the training data. In other words, if K = N-1, we can perfectly match all data points with a polynomial.

1.3.4 different subset

In here, we see that some issues may arise due to:

- linear dependence between the vectors. In this case, the matrix $\mathbf{X}$ becomes degenerate and we can no longer invert it.
- numerical issues with pseudo inversion. As we set up the

  `fit_multiple_lin_model`

  function to use a pseudoinverse, instead of a true matrix inverse, we don't necessarily find the ideal solution.
- Further, the data may not fit a degree n polynomial. If it doesn't and we choose sufficiently disparate data points, any degree n polynomial model will fail to properly match or approximate our function.

2 Logistic Regression

Design matrices:

- Data set 1: since the boundary looks like part of a unit circle, centered on (30,30) and a unit circle in 2D is sufficiently approximable by the L2 norm, I will use two features $[x_1, x_1^2, x_2, x_2^2]$.
- Data set 2: This boundary appears to look like a quadratic function, I will use $[x_1^1, x_2^1, x_1^2, x_2^2, x_1 * x_2]$ ie. any option of order two or below.
- Data set 3: this boundary roughly resembles the graph of f(x) = a x^3, so I will use any feature combination that goes up to order three $[x_1, x_2, x_1^2, x_2^2, x_1 * x_2, x_1^2 * x_2, x_1 * x_2^2, x_1^3, x_2^3]$

2.1 programming task

2.2 programming task

2.3 programming task

2.4 parameters

Data set 1:

- Bias = -55.797
- Weights = [3.089, 2.822, -0.053, -0.044]
- Train accuracy: 99.846 %
- Test accuracy: 98.765 %
- Train loss: 0.022
- Test loss: 0.044

Data set 2:

- Bias = -37.283
- Weights = [5.186, $8.421*10^{-1}$, $-1.808*10^{-1}$, $9.597*10^{-4}$]
- Train accuracy: 100 %
- Test accuracy: 100 %
- Train loss: 0.046
- Test loss: 0.044

Data set 3:

- Bias = 1.140
- Weights = [-5.721, 3.501, -2.803, -0.448, -0.180, -1.167. 0.256 , 3.550, 0.111]
- Train accuracy: 97.292 %
- Test accuracy: 95.833 %
- Train loss: 0.092
- Test loss: 0.126

3 Gradient descent

3.1 programming task

3.2 programming task

3.3 derivatives

- $\frac{\partial f(x,y)}{\partial x} = \nabla_x(1+\frac{x^2+y^2}{4000} - cos(x)cos(\frac{y}{\sqrt{2}})) = \frac{2x}{4000} + sin(x)cos(\frac{y}{\sqrt{2}}) = \frac{x}{2000} + sin(x)cos(\frac{y}{\sqrt{2}})$
- $\frac{\partial f(x,y)}{\partial y} = \nabla_x(1+\frac{x^2+y^2}{4000} - cos(x)cos(\frac{y}{\sqrt{2}})) = \frac{2y}{4000} + \frac{cos(x)sin(\frac{y}{\sqrt{2}})}{\sqrt{2}} = \frac{y}{2000} + \frac{cos(x)sin(\frac{y}{\sqrt{2}})}{\sqrt{2}}$

3.4 programming task

3.5 definition of partial derivatives

- $\frac{\partial f}{\partial x} = lim(h->0)\frac{f(x+h,y)-f(x-h,y)}{2h}$
- $\frac{\partial f}{\partial y} = lim(h->0)\frac{f(x,y+h)-f(x,y-h)}{2h}$

Result of implementing the finite difference mthod:

- True gradient: $\frac{\partial f(x_0,y_0)}{\partial x}$ = -0.8239220888918439, $\frac{\partial f(x_0,y_0)}{\partial y}$ = 0.04104398036631918
- finite difference gradient: "$\frac{\partial f(x_0,y_0)}{\partial x}$" = -0.823922088866169, "$\frac{\partial f(x_0,y_0)}{\partial x}$" = 0.04104398036997026

The approximation with h set to 10^{-5} matches the exact solution up to an order of 10^{-11} in the $\frac{\partial f(x_0,y_0)}{\partial x}$ case and 10^{-12} in the $\frac{\partial f(x_0,y_0)}{\partial y}$ case, which suggests that the finite difference method works as a good approximator here.

3.6 Why is this function hard to optimize

Since the largest amplitude component of the function is a product of two cosine functions our two independent variables (they y scaling doesn't affect this behaviour strongly), there are lots of peaks and valleys ordered on a grid. The gradient descent will have a hard time picking out the underlying gradient towards (0,0) from the much higher "impact" gradient contained in the cosine produce. in other words, the gradient can be seen as a linear combination of two simpler gradients, one being scaled at $\frac{1}{2000}$ when compared to the other. Naturally, the downscaled gradient will look comparatively unimportant/have a small impact on gradient descent, which has much higher descent directions stemming from sine and cosine curves.

3.7 hyperparameter optimization

After doing a hyperparameter search, the chose parametrization was

- learning rate = 0.001
- learning rate decay = 10^{-5}
- number of epochs = 10^5

which yielded a local optimum f(3.1397, 4.4359) = 0.0074.

3.8 2D contour plot

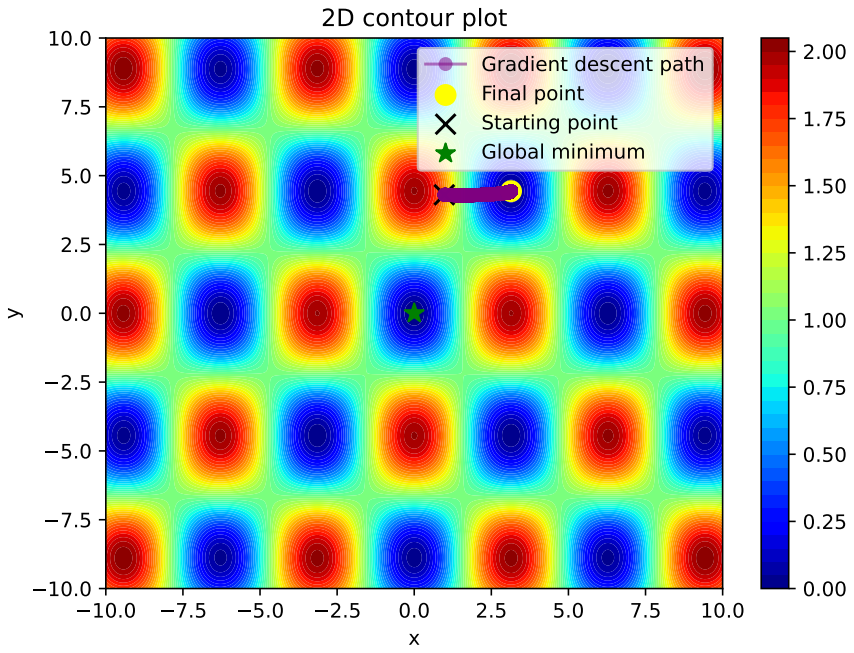


Figure 3: 2D contour plot of the gradient descent method

As we can see, the gradient descent path follows the steeper direction of the cosine functions, while not being provided sufficient epochs to see the small underlying gradient of the quadratic function. The algorithm then gets stuck in a local minimum, since the total descent direction point inwards so to speak, even though there is a larger global minimum. One way that this can be avoided is by adding a heat parameter, that sometimes randomly jostles the position vector around and then optimizing over multiple gadient descent methods.

3.9 iterative change of f

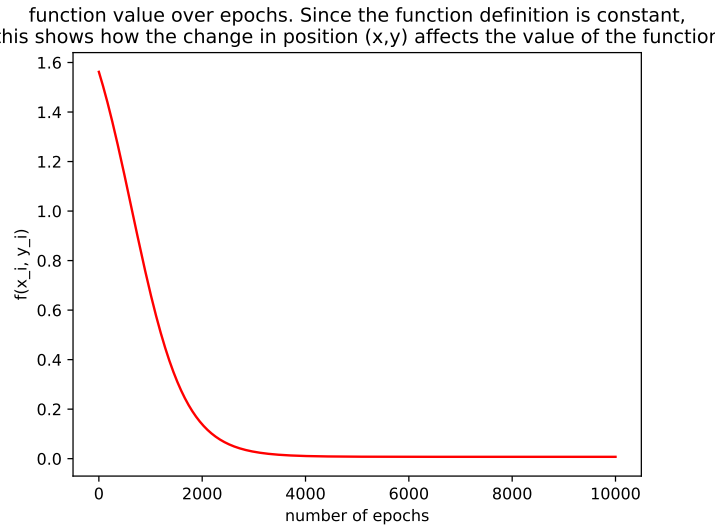


Figure 4: griewak function evaluated at each epoch for its corresponding (x,y) pair

As one can see in this image, the major optimization occurs in the first roughly 2500 epochs. Afterwards there is no strong improvement. This effect can be either due to the learning rate decaying so fast, the function can no longer meaningfully change, or due to being stuck at a local minimum. The third option, which would not be a problem, is due to the function reaching the global minimum where the derivative is zero.

3.10 learning rate discussion

if the learning rate decay is set to 1, in this exercise, we will keep a constant learning rate throughout all epochs. this means the function will quickly converge as close as it can to the minimum it finds. However, after the local (or global) minimum is closer than one step size (=the learning rate), the method will now bounce around the minimum in various directions without actually getting closer. This is the whole idea behind the learning rate decay: Once a reasonable approximation of the location of a minimum is made, refine the search by decreasing the step size (graphically, this could be seen as drawing a finer mesh over the area of interest).